The Relay: Production RAG Technical Report

1 What I built and why

The pipeline has five steps: analyse the question, retrieve passages (a tool call that can time out), decide whether the evidence is enough, draft a cited answer, and check the citations in code. The decision that shaped it most was verifying before streaming. The first token waits for the whole draft (median 638 ms in the real run), but wrong answers stay at 0 to 6.7%. The price is that 20 to 27% of answerable questions are refused.

2 The real service and my stand-in

I evaluated Groq serving `qwen/qwen3.8-27b`. Probing identified binding limits: 8,000 TPM (`x-ratelimit-limit-tokens: 8000`) and 1,000 OTPM (`Limit 1000, Used 970, Requested 120`). A live 429 error exposed a 200,000 TPD ceiling (`Limit 200000, Used 199180, Requested 2032`), refilling continuously at ~2.31 tokens/s (0.432 s/token), not a midnight reset. Concurrency sustained 4 simultaneous calls (~210 ms). Costs assume Groq's list prices ($0.80/1M prompt, $4.00/1M completion tokens). To simulate load without quota exhaustion, I built a calibrated stand-in enforcing 8,000 TPM and 1,000 OTPM (~0.12 ms/tok prefill, ~7.5 ms/tok decode, 1.25^{N-1} queue penalty) with 4x virtual time acceleration.

3 Results

Across six regimes (Tables 2a, 2b): Under `steady_real` (3.1 req/min offered vs ~3.0 capacity), 13 of 20 requests (65%) hit TPM/OTPM throttles; all 13 retried attempts hit 429 again. Answered queries had median TTFT of 638.4 ms (total: 1,045.0 ms). In stand-in simulations, `steady` achieved 100% completion (276.9 ms TTFT). In `burst` (20 arrivals), 9 requests (45%) succeeded while 11 timed out (peak queue 20). Under `sustained_overload` (~3x capacity), queue peaked at 23, admitting 36.4% while shedding 63.6% via timeout. Fast 429 rejections (~28–35 ms) on dropped queries artificially deflated unconditioned percentiles; isolating `status=ok` rows revealed true TTFT (581.3 ms burst, 259.7 ms overload). Daily quota exhaustion returned fast refusals in 46.0 ms (stand-in) and 65.0 ms (real).

4 Where the time goes

Client retries dwarf model latency. In `pipe-probe-4`, first-token latency reached 7,150.8 ms (total: 7,325.7 ms) despite raw generation taking only 352.0 ms:

- **Attempt 0 (429)**: Rejection in 111.9 ms; 2,455.6 ms backoff sleep.
- **Attempt 1 (429)**: Rejection in 57.1 ms; 4,159.4 ms backoff sleep.
- **Attempt 2 (200)**: Headers in 108.5 ms; raw TTFT 269.4 ms; generation in 352.0 ms.
- **Deterministic Verification**: Finished in 0.41 ms.

Because verification gates token streaming, user-visible first-token latency was 7,150.8 ms ($111.9 + 2455.6 + 57.1 + 4159.4 + 352.0 + 0.41 + 14.4\text{ retrieval}$).

5 When a limit is hit

- **Per-Minute Limits (8,000 TPM / 1,000 OTPM)**: Requests queue up to a 15s patience deadline. Spikes are delayed by sliding-window refills (2–12s); calls exceeding 15s are dropped with a timeout notice.
- **Per-Day Limits (200,000 TPD)**: Rolling refill (~2.31 tokens/s) requires ~8.7m clearance. Requests cannot wait and are rejected immediately with a refill estimate ("try again in about 9 minutes").
- **Dynamic Degradation**: Not implemented; pipeline does not dynamically downscale chunk sizes under quota pressure.

Retrieval: quality versus cost

Evaluating three settings across the common 15-question cohort (Q1–Q15, n=15): Setting 1 (`small_k3_norerank`) matched Setting 2 (`medium_k4_norerank`) in accuracy (53.3% correct and cited, 20.0% correctly refused) while consuming only 896.5 prompt tokens/answer vs 2,195.5. Small pieces achieved parity at 40.8% of the input cost. Across all 20 questions, Settings 1 and 2 matched at 40.0% correct and 35.0% refused, with Setting 1 using 40.5% of prompt tokens ($894.7 / 2,209.3$). In Setting 3 (`large_k3_rerank`), Q16–Q20 were unserved because heavy context (2,331.7 input tokens) hit the 200k daily ceiling; its comparison is partial.

How this library will mislead a naive retriever

1. **Bibliography Bleed**: Querying "Attention Is All You Need" matched Moshi's bibliography (pages 62–65) due to dense keyword citations, displacing genuine 2017 Transformer content. Bibliographies must be stripped.
2. **Split Facts Across Pages**: In Q3, Moshi latency (160ms/200ms) appears on Pages 1–2, but Mimi codec specs (12.5 Hz, 1.1 kbps) appear on Pages 11 and 23. Local retrieval captures one page and hallucinates or refuses the other.
3. **Cross-Paper Synthesis (Q16)**: Comparing Transformer heads (8) with Moshi Depth Transformer (16) requires retrieval across disjoint documents; single queries bias toward one paper.

6 Retries

Across three trials in `retry_experiments.jsonl`, waiting exactly `retry-after` resulted in 6 of 12 requests (50.0%) hitting 429 again. Jittered backoff (+0.1 to 1.5s) resulted in 8 of 12 requests (66.7%) hitting 429 again; the window was too narrow to absorb the 1,000 OTPM clearance. In `steady_real`, 13 of 13 retried attempts (100%) hit 429 again because offered rate (~3.1 req/min) exceeded capacity (~3.0 req/min).

7 What the person sees

At 3 pm with the daily budget gone, the reply arrives in about 65 ms: "The research paper assistant has reached its daily compute limit (200,000 tokens/day rolling bucket refilling at ~2.3 tokens/s). Please try again in about 9 minutes." There are no retries and no guessed answer. The wait time (9 minutes) is dynamically computed from the actual 429 response retry-after value (524s), because the daily limit refills continuously at ~2.3 tokens/s rather than at midnight.

8 What I'm unsure about, what I left out, what I learned

Least certain: the stand-in's queueing penalty (1.25^(N-1)) and 4x time scaling. I tested only 4 concurrent calls on the real service, so burst and overload results show the shape of the behaviour, not its exact size. Not built: a check that a cited passage really supports the claim (my verifier checks pages and word overlap only), and one search per paper for cross-paper questions. New to me: a "per day" limit can be a rolling bucket. One 429 said 1,212 tokens short, retry in 8m43.584s, and 1,212 x 86,400/200,000 s = 523.584 s. It refills continuously, so midnight never matters.

Your feedback on Vaani

What stood out: concrete numbers (~450 ms p90) and Teacher AI asking me to review a prompt diff before saving. What confused me: the latency figure doesn't say what is timed, and "Try the platform" and "Start free" point to different domains. I'd define the metric and show p50, p95 and p99, because a p90 hides the slow calls where callers hang up.

How I worked with the assistant

I built with Antigravity in approval-gated stages and used a second assistant (Claude chat) to review each stage before I approved it. Both full histories are attached (conversation_antigravity.md, conversation_claude_review.md). Claude also drafted my test questions and Vaani notes; I checked every expected answer against the paper text. Three prompts that mattered most: (1) plan before any code, using the brief's tables and CSV columns as a checklist; (2) set max_retries=0 and log every attempt, because a 12.7 s first-token time had looked like Groq's queue and the real figure was 438 ms; (3) show the exact sentence and page for every expected answer, which caught a wrong expected answer (Q4) before it distorted Table 3. Caught: the assistant said 99.9% of the delay was inside Groq's queue without measuring it; the retries-off run showed the SDK had been silently retrying 429s. Overruled: its plan ranked "nobody turned away" last and rejected overflow with a 503; I replaced that with a deadlined queue and a "try again in N minutes" reply. Distinctly mine: running everything on my own key, stopping to re-verify each stage instead of accepting summaries, and deciding what to correct or cut when the numbers disagreed.

Benchmark Evidence & Operational Tables

Table 1: Limits you found in the real service

Limit	Value you found	How you found it	Confidence
Requests per minute	Not measured	Empirically tested; binding limits are TPM/OTPM. Docs cite 30 RPM for other models.	Low
Tokens per minute (Total TPM)	8,000 TPM	Live HTTP response header: x-ratelimit-limit-tokens: 8000 (probe_raw.jsonl)	High
Output tokens per min (OTPM)	1,000 OTPM	Live HTTP 429 body: 'Limit 1000, Used 970, Requested 120' (probe_raw.jsonl)	High
Requests per day (RPD)	1,000 req/day	Live response header x-ratelimit-limit-requests: 1000 (probe_raw.jsonl)	High
Tokens per day (TPD)	200,000 tok/day	Live HTTP 429 body: 'Limit 200000, Used 199180, Requested 2032'. Rolling leaky bucket (~2.3 tok/s), NOT midnight.	High
Requests in flight at once	>= 4 concurrent	Empirically tested 4 simultaneous requests; all 200 OK in ~210ms (retry_experiments.jsonl).	Medium
What it returns on limit	HTTP 429	JSON body with type: tokens, code: rate_limit_exceeded (probe_raw.jsonl)	High
Time until it recovers	3 to 6 s (median 6s), up to 12s	probe_raw.jsonl shows retry-after of 3 to 6 s (median 6 s) and retry_experiments.jsonl shows waits up to 12 s	High

Table 2a: Latency, one row per load scenario (status=ok rows only)

Scenario	Type	Reqs	Dur (s)	Offered Rate	First token p50/p95/p99 (ms)	Total time p50/p95/p99 (ms)
steady_real	real	20	382.7	3.1/min (capacity ~3.0)	638.4 / 1108.8 / 1247.0	1045.0 / 1613.5 / 1614.8
probe_pipeline	real	12	144.6	5.0/min	968.1 / 7348.4 / 7433.5	2369.5 / 7660.4 / 7804.6
steady	stand-in	12	66.6	2.7 virt (10.8 real)	276.9 / 337.0 / 356.0	577.1 / 612.9 / 622.6
burst	stand-in	20	1.9	158.7 virt (634.7 real)	581.3 / 933.9 / 1028.5	888.8 / 1241.4 / 1336.0
sustained_overload	stand-in	33	45.1	11.0 virt (43.9 real)	259.7 / 340.7 / 364.3	567.2 / 648.2 / 671.8
limit_halved	stand-in	12	66.6	2.7 virt (10.8 real)	275.5 / 333.9 / 347.7	577.2 / 601.2 / 607.8
daily_budget_exhausted	stand-in	10	1.0	150.0 virt (600 real)	n/a	n/a
daily_budget_exhausted_real	real	1	1.0	60.0/min	n/a	n/a

Table 2b: Outcomes, same rows

Scenario	Succ %	429 %	Drop %	Drop Wait p50	Graceful reply p50	Retries per req	Peak Q	Tokens (in/out/tot)	Cost/1k Ans
steady_real	35.0%	65.0%	0.0%	-	-	0.65	1	2129 / 54 / 2183	$1.918
probe_pipeline	66.7%	33.3%	0.0%	-	-	1.0	1	2078 / 104 / 2182	$2.078
steady	100.0%	0.0%	0.0%	-	-	0.0	1	2164 / 37 / 2200	$1.878
burst	45.0%	0.0%	55.0%	30.9 ms	-	0.0	20	2097 / 38 / 2135	$1.830
sustained_overload	36.4%	0.0%	63.6%	27.5 ms	-	0.0	23	2156 / 34 / 2190	$1.862
limit_halved	66.7%	0.0%	33.3%	27.2 ms	-	0.0	1	2156 / 34 / 2191	$1.863
daily_budget_exhausted	0.0%	100.0%	0.0%	-	46.0 ms	0.0	1	0 / 0 / 0	n/a
daily_budget_exhausted_real	0.0%	100.0%	0.0%	-	65.0 ms	0.0	1	0 / 0 / 0	n/a

Table 3: Retrieval settings (Common n=15 benchmark cohort)

Setting	n	Correct %	Wrong %	Refused OK %	Refused Wrong %	Avg In	Avg Out	Total Tok	First Token p50/p95	Total p50/p95
small_k3_norerank	15	53.3%	6.7%	20.0%	20.0%	896.5	72.9	969.4	766.5 / 3456.1 ms	1438.7 / 4082.4 ms
medium_k4_norerank	15	53.3%	0.0%	20.0%	26.7%	2195.5	66.0	2261.5	947.8 / 1432.3 ms	1318.0 / 2336.6 ms
large_k3_rerank	15	46.7%	6.7%	20.0%	26.7%	2331.7	80.9	2412.6	898.5 / 1700.2 ms	1120.0 / 2269.8 ms

Note on Setting 3: Questions Q16–Q20 in Setting 3 were not answered because heavy context (2,331.7 input tokens) exhausted the real 200,000 tokens/day limit. Comparison is strictly over common n=15 questions.
Full 20-question benchmark (Settings 1 & 2): Setting 1 achieved 40.0% Correct, 5.0% Wrong, 35.0% Refused OK (894.7 in / 67.0 out). Setting 2 achieved 40.0% Correct, 0.0% Wrong, 35.0% Refused OK (2,209.3 in / 66.5 out). Parity achieved at 40.5% prompt token cost.